



SPARTAN INTERNATIONAL SCHOOL

SR. SEC. SCHOOL

CHENNAI - 600 122

COMPUTER SCIENCE PROJECT

ON

3D 3-DOF ROBOT ARM

Kinematics, MuJoCo Simulation and PID Control

Name	:	Joel Jebitto O
Class	:	XII
Subject	:	Computer Science
Academic Year	:	2025-2026
School	:	Spartan International School

2. Acknowledgement

I, **Joel Jebitto O**, a student of **Class XII** at **Spartan International School**, would like to express my sincere gratitude to my Computer Science teacher and my school for giving me the opportunity to prepare this project report.

I am thankful for the guidance, encouragement, and support received during the development and documentation of this project. This project helped me understand how programming, mathematics, simulation, and control systems can be combined to create a working interactive application.

I would also like to thank my parents, classmates, and online technical documentation resources for their support. Through this work, I learned how a software project can be organized into modules, how a graphical interface communicates with mathematical logic, and how robotics concepts such as forward kinematics, inverse kinematics, simulation, and optional PID control can be explained in a clear and practical manner.

Prepared By	School
Joel Jebitto O	Spartan International School

3. Index / Table of Contents

Index / Table of Contents

1. Cover Page	1
2. Acknowledgement	2
3. Index / Table of Contents	3
4 Introduction	5
4.1 Project Overview	5
4.2 Problem Solved	5
4.3 Usefulness	5
4.4 Where It Can Be Applied	5
5 Project Objectives	6
6 Problem Statement	7
7 Proposed Solution	8
7.1 Solution Summary	8
7.2 Main Design Choices	8
7.3 Recommended Architecture	8
8 Technology Stack	9
9 System Requirements	10
9.1 Hardware Requirements	10
9.2 Software Requirements	10
10 File Structure	11
10.1 Important Files and Folders	12
11 System Architecture	13
11.1 Layered Architecture	13
11.2 Text-Based Architecture Diagram	13
11.3 Data Flow	14
11.4 Non-Functional Requirements	15
12 Algorithm and Logic	16
12.1 Core Algorithm Summary	16
12.2 Flowchart-Style Explanation	16
12.3 Pseudocode	17
12.4 Important Decision Points	17
12.5 Input and Output Flow	17
13 Mathematical Explanation	18
13.1 Concept: Joint Angles and Radians	18
13.2 Concept: Robot State Vector	18
13.3 Concept: Forward Kinematics	18
13.4 Concept: Link Position Geometry	19
13.5 Concept: Jacobian Matrix	20
13.6 Concept: Inverse Kinematics Error	21
13.7 Concept: Damped Least-Squares Inverse Kinematics	21
13.8 Concept: Workspace and Safety Limits	21
13.9 Concept: PID Control	22
13.10 Concept: Simulation Time Step	22
13.11 Concept: Algorithmic Complexity	23
14 UI Explanation	24
14.1 Main UI Layout	24
14.2 Forward Kinematics Tab	24
14.3 Inverse Kinematics Tab	25
14.4 PID Control Tab	26
14.5 How UI Events Connect to Backend Logic	27
15 Main Logic Code Explanation	28
15.1 Application Entry Point	28
15.2 Configuration Values	28
15.3 Safety Validation	28
15.4 Inverse Kinematics Result Object	29
15.5 MuJoCo Model Creation	29
15.6 PID Simulation Step	30
16 Demo / Program Run-through	31
16.1 Setup Commands	31

16.2	Step-by-Step Walkthrough	31
16.3	MuJoCo Simulation Output	32
16.4	Backend Output During Demo	32
16.5	What Happens Internally During the Demo	32
17	Results and Discussion	33
17.1	Final Output of the Project	33
17.2	Objective Achievement	33
17.3	Strengths of the Implementation	33
17.4	Limitations of the Current Version	33
17.5	Possible Improvements	33
18	Future Applications	34
19	Conclusion	35
20	Appendix	36
20.1	Setup Notes	36
20.2	Key Constants from Configuration	36
20.3	Additional Code Snippet: Setup Script	36
20.4	Additional Diagram: Control Data Flow	37
21	Bibliography / References	38

4. Introduction

4.1 Project Overview

This project mainly focuses on **forward kinematics** and **inverse kinematics** for a 3D 3-DOF robot arm. The application allows the user to calculate the end-effector position from joint angles and solve the required joint angles for a desired target position. PID control is included only as an additional feature to demonstrate smooth joint-space motion in simulation.

The Python application combines:

- a **Tkinter graphical user interface** for user input and live output,
- **forward kinematics** to calculate end-effector position from joint angles,
- **inverse kinematics** to calculate joint angles from a target position,
- a **MuJoCo simulation model** to display the robot arm in 3D, and
- an **optional PID controller** to show smooth feedback-based movement after a target pose is selected.

The robot has three revolute joints: base yaw, shoulder pitch, and elbow pitch. These joints control the Cartesian position of the end effector, represented as $\mathbf{p} = [x, y, z]^T$. The project does not control full 3D orientation because that would require additional degrees of freedom.

4.2 Problem Solved

Robotics concepts are often difficult to understand when they are only shown as formulas. This project solves that learning problem by connecting formulas to a working program. A student can move sliders, see the numerical output, and view the robot pose in simulation.

4.3 Usefulness

The project is useful because it explains the full path from mathematical theory to working software. It demonstrates how the main FK and IK calculations are separated from the user interface, visualization, optional control logic, and simulation output.

Project Identity

Main project	3D 3-DOF MuJoCo Kinematics and PID Demo
Primary language	Python
Main GUI	Tkinter with <code>ttk</code> widgets
Simulation	MuJoCo physics engine
Core ideas	Forward kinematics, inverse kinematics, Jacobian, safe workspace limits, and optional PID motion

4.4 Where It Can Be Applied

This type of project can be applied in robotics education, robotic arm prototyping, control systems demonstrations, simulation-based learning, and early-stage automation experiments. The same concepts are used in industrial robot manipulators, pick-and-place machines, robotic welders, medical robots, and laboratory automation systems.

5. Project Objectives

The main objective of this project is to demonstrate forward kinematics and inverse kinematics for a 3-DOF robot arm using a Python-based graphical and simulation environment. PID control is included as an additional feature to demonstrate smooth joint-space motion.

The specific objectives are:

1. To build a Python application that demonstrates a three-joint robot arm.
2. To calculate end-effector position from joint angles using forward kinematics.
3. To calculate suitable joint angles for a target position using inverse kinematics.
4. To visualize FK and IK results clearly using Tkinter and MuJoCo.
5. To include PID control as a secondary feature for smooth joint-space motion.
6. To provide a clean Tkinter interface with sliders, input boxes, buttons, status messages, and output panels.
7. To keep the project modular so that UI, math, optional control, and simulation code are easy to understand.
8. To explain the mathematical and computational logic in a Class XII-level report.

Objective	Expected Learning Outcome
Forward kinematics	Understand how joint angles produce an x, y, z position.
Inverse kinematics	Understand how a desired position can be converted into joint angles.
PID control (<i>additional</i>)	Understand how feedback can make target motion smoother after FK or IK selects a pose.
Simulation	Understand how software can model a physical robot before hardware is built.
UI design	Understand how users interact with backend logic through controls and visual feedback.

6. Problem Statement

A robotic arm must move its end effector to a useful point in space. However, the computer usually controls joints, not the end-effector position directly. This creates two linked problems:

- Given joint angles q_1, q_2, q_3 , calculate where the end effector is located.
- Given a target point (x, y, z) , calculate joint angles that can reach it safely.

A secondary problem is motion quality. Directly setting joint angles makes the simulated robot immediately change pose. The project therefore includes PID control as an additional feature to show how feedback can create smoother joint-space motion, while FK and IK remain the main problems addressed.

Formal Problem Definition

Given a target position $\mathbf{p}_{target} \in \mathbb{R}^3$, find a joint vector $\mathbf{q} = [q_1, q_2, q_3]^T$ such that:

$$\|\mathbf{p}_{target} - f(\mathbf{q})\| < \epsilon$$

while obeying joint limits and keeping all moving robot points above the floor safety limit.

7. Proposed Solution

The proposed solution is a modular Python application with separate components for interface, mathematics, control, and simulation.

7.1 Solution Summary

The user enters desired joint angles or Cartesian target values through the Tkinter UI. The backend validates the input, applies safety limits, and then runs the appropriate mode. The main operating modes are Forward Kinematics Mode and Inverse Kinematics Mode. PID Control Mode is included as an additional feature to demonstrate smooth motion after a target pose is selected.

1. **Forward Kinematics Mode:** user changes joint angles and the program calculates the end-effector position.
2. **Inverse Kinematics Mode:** user selects a target point and the solver calculates suitable joint angles.
3. **Additional PID Control Mode:** user selects a target joint pose and the controller generates torques to move the robot smoothly.

7.2 Main Design Choices

Choice	Reason	Trade-off
Tkinter UI	Built into Python installations and suitable for school projects.	Less modern than web or Qt UI frameworks.
MuJoCo simulation	Provides real 3D model, joints, actuators, gravity, and viewer support.	Requires OpenGL/display support.
Damped least-squares IK	More stable near difficult positions than a pure inverse Jacobian.	Iterative and may fail outside the safe workspace.
PID joint control	Added as an explainable extension for smooth target motion after FK or IK pose selection.	Requires gain tuning and torque limits.

7.3 Recommended Architecture

The best architecture for this school project is a **layered desktop architecture**. It is simpler than a web application and more practical than a hardware-only setup.

- **Cost:** low, because it uses Python and open-source libraries.
- **Complexity:** moderate, because MuJoCo and control logic are separated into modules.
- **Scalability:** can be extended with more joints, improved UI, or database logging.
- **Operations:** runs locally with a virtual environment and does not require a server.

8. Technology Stack

Component	Technology	Use in Project
Programming language	Python	Main language for UI, FK/IK mathematics, optional PID control, simulation setup, scripts, and tests.
GUI framework	Tkinter and ttk	Builds the desktop interface, tabs, sliders, entries, buttons, and status bar.
Numerical computing	NumPy	Handles vectors, matrices, trigonometry, clipping, norms, and linear algebra.
Visualization in UI	Matplotlib	Renders equations in the UI and draws the live PID response plot.
Simulation engine	MuJoCo	Loads the robot model, creates joints and actuators, steps physics, and displays the 3D arm.
Configuration	<code>config.py</code>	Stores link lengths, joint limits, optional PID gains, UI constants, and simulation timestep.
Scripts	Bash and Python	Provide setup and run helpers through <code>scripts/setup.sh</code> and <code>scripts/run.sh</code> .
Dependencies	<code>requirements.txt</code>	Lists <code>mujoco>=3.3.0</code> , <code>numpy>=2.0</code> , <code>matplotlib>=3.9</code> , and <code>pytest>=8.0</code> .
Testing tools	pytest	Used in the repository for math, controller, simulation, import, and UI component checks.
Development tools	GitHub, Git, VS Code or any Python IDE	Repository hosting, source management, and local development.

```

1 mujoco>=3.3.0
2 numpy>=2.0
3 matplotlib>=3.9
4 pytest>=8.0

```

Listing 1: requirements.txt - project dependencies

9. System Requirements

9.1 Hardware Requirements

Hardware	Recommended Requirement
Processor	64-bit dual-core processor or better. A quad-core CPU is recommended for smoother simulation and UI response.
RAM	Minimum 4 GB; 8 GB or more recommended.
Storage	Around 500 MB to 1 GB free space for repository, virtual environment, and Python packages.
Display	1366 x 768 minimum; 1920 x 1080 recommended for comfortable viewing of all tabs and output panels.
GPU / Graphics	Integrated or dedicated GPU with working OpenGL support. This is important for the MuJoCo viewer.
Internet	Required for cloning the repository and installing dependencies for the first time. Not required after setup.

9.2 Software Requirements

Software	Requirement
Operating system	Windows, Linux, or macOS. Ubuntu/Linux is convenient because Tkinter can be installed using <code>python3-tk</code> .
Python runtime	Python 3.10 or newer recommended.
Python packages	<code>mujoco</code> , <code>numpy</code> , <code>matplotlib</code> , and <code>pytest</code> .
Tkinter	Required for the GUI. On Ubuntu, install using <code>sudo apt install python3-tk</code> .
MuJoCo dependencies	Working display server and OpenGL driver for the viewer.
IDE / Editor	VS Code, PyCharm, IDLE, or any text editor.
Git	Recommended for cloning and version control.
Browser	Useful for reading documentation and GitHub repository pages.
Database	Not required in the current version. Future versions may add SQLite or cloud storage.

10. File Structure

The file structure below is based on the actual GitHub repository contents that were read during report preparation. The project is separated into folders by responsibility.

```
School_cs_project_12th/
├── assets/
│   └── robot_motion.gif
├── control/
│   ├── __init__.py
│   └── pid.py
├── kinematics/
│   ├── __init__.py
│   ├── common.py
│   ├── forward.py
│   └── inverse.py
├── scripts/
│   ├── run.sh
│   ├── run_project.py
│   ├── setup.sh
│   └── setup_project.py
├── simulation/
│   ├── __init__.py
│   └── mujoco_sim.py
├── tests/
│   ├── test_app_pid_behavior.py
│   ├── test_controller.py
│   ├── test_equation_rendering.py
│   ├── test_main_imports.py
│   ├── test_mujoco_model.py
│   ├── test_robot_math.py
│   ├── test_simulation.py
│   └── test_slider_control.py
├── ui/
│   ├── __init__.py
│   ├── app.py
│   ├── fk_actions.py
│   ├── forward_tab.py
│   ├── ik_actions.py
│   ├── inverse_tab.py
│   ├── layout.py
│   ├── pid_actions.py
│   ├── pid_tab.py
│   ├── style.py
│   └── widgets.py
├── .gitignore
├── README.md
├── REPORT.md
├── config.py
├── main.py
└── requirements.txt
```

10.1 Important Files and Folders

File / Folder	Purpose
main.py	Entry point. Starts the Tkinter application and handles missing dependency messages.
config.py	Stores all important constants: link lengths, joint limits, optional PID gains, simulation timestep, UI colors, and default targets.
kinematics/forward.py	Implements forward kinematics, robot joint positions, and the position Jacobian.
kinematics/inverse.py	Implements inverse kinematics, geometric seed selection, workspace checks, and safety validation.
control/pid.py	Implements the joint-space PID controller and the history buffer for plotting.
simulation/mujoco_sim.py	Builds the MuJoCo MJCF model and manages the model, data, viewer, gravity, and simulation stepping.
ui/app.py	Coordinates UI state, simulation state, PID state, callbacks, and the main update loop.
ui/layout.py	Builds the main window structure: top bar, tabs, and status bar.
ui/forward_tab.py	Builds the Forward Kinematics tab controls and output panel.
ui/inverse_tab.py	Builds the Inverse Kinematics target controls, solver buttons, and output panel.
ui/pid_tab.py	Builds the PID target controls, gain controls, live values, and response plot.
ui/widgets.py	Reusable UI components such as sliders, equation boxes, output text panels, and scrollable frames.
scripts/setup.sh and scripts/setup_project.py	Create the virtual environment and install dependencies.
scripts/run.sh and scripts/run_project.py	Run the application using the project virtual environment.
tests/	Contains repository checks for robot math, controller behavior, MuJoCo model loading, UI components, and imports.

11. System Architecture

11.1 Layered Architecture

The project follows a layered architecture. The central path is built around forward kinematics and inverse kinematics, while the Tkinter interface and MuJoCo viewer make these calculations visible and interactive. Optional PID control is kept as a secondary smooth-motion feature.

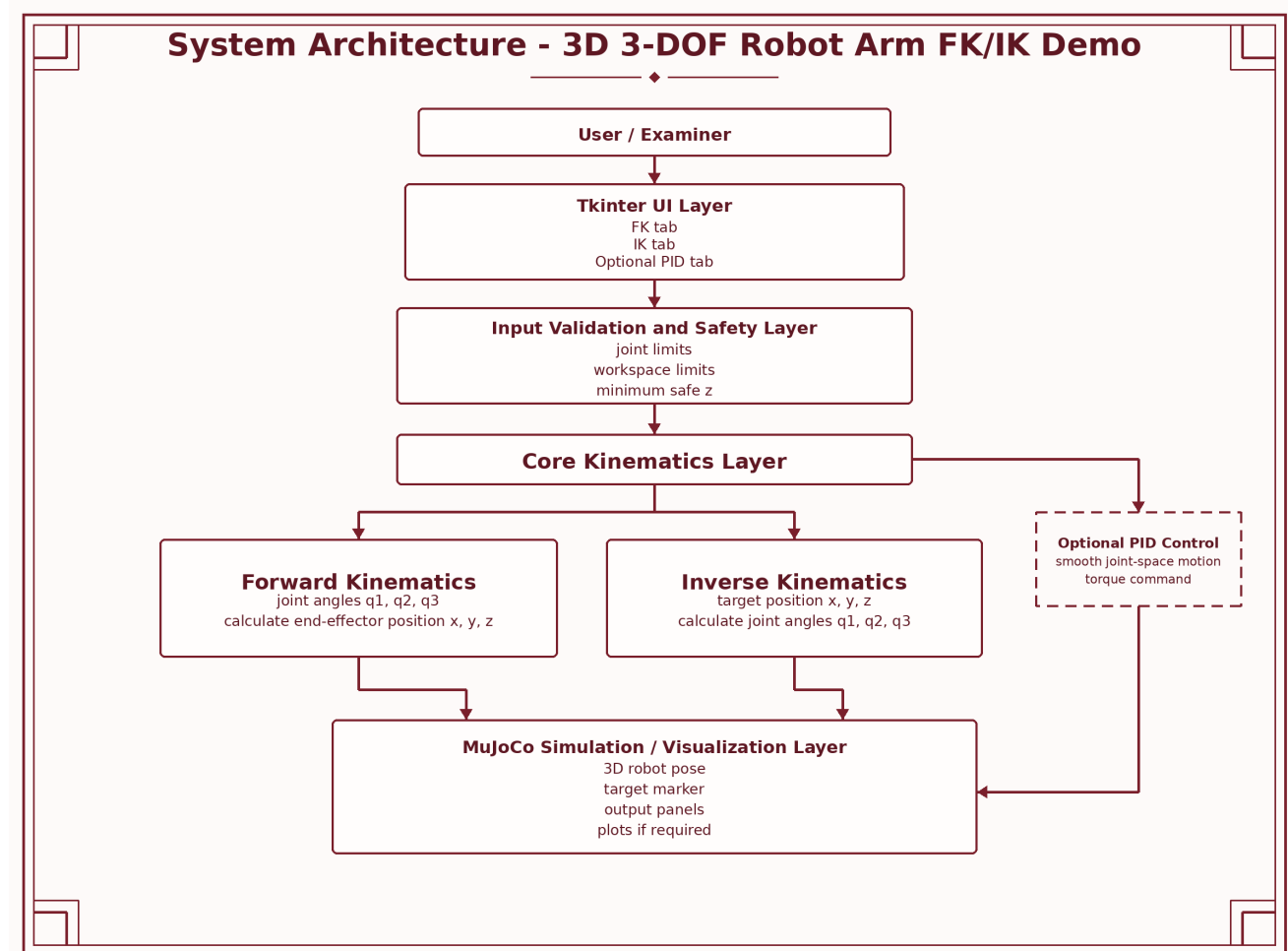
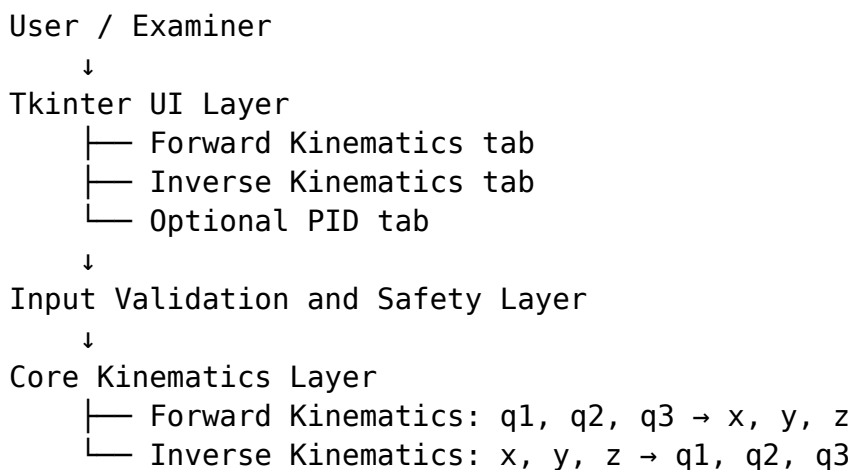


Figure 1: System architecture for the 3D 3-DOF robot arm FK/IK demo, with PID shown as an optional smooth-motion feature.

11.2 Text-Based Architecture Diagram



↓
MuJoCo Simulation / Visualization Layer

↓
Output: robot pose, target marker, text panels, optional plots

11.3 Data Flow

Data flows from user input to validation and then into the selected FK or IK workflow. Optional PID motion is shown as a secondary path that can smooth movement after a target pose is selected.

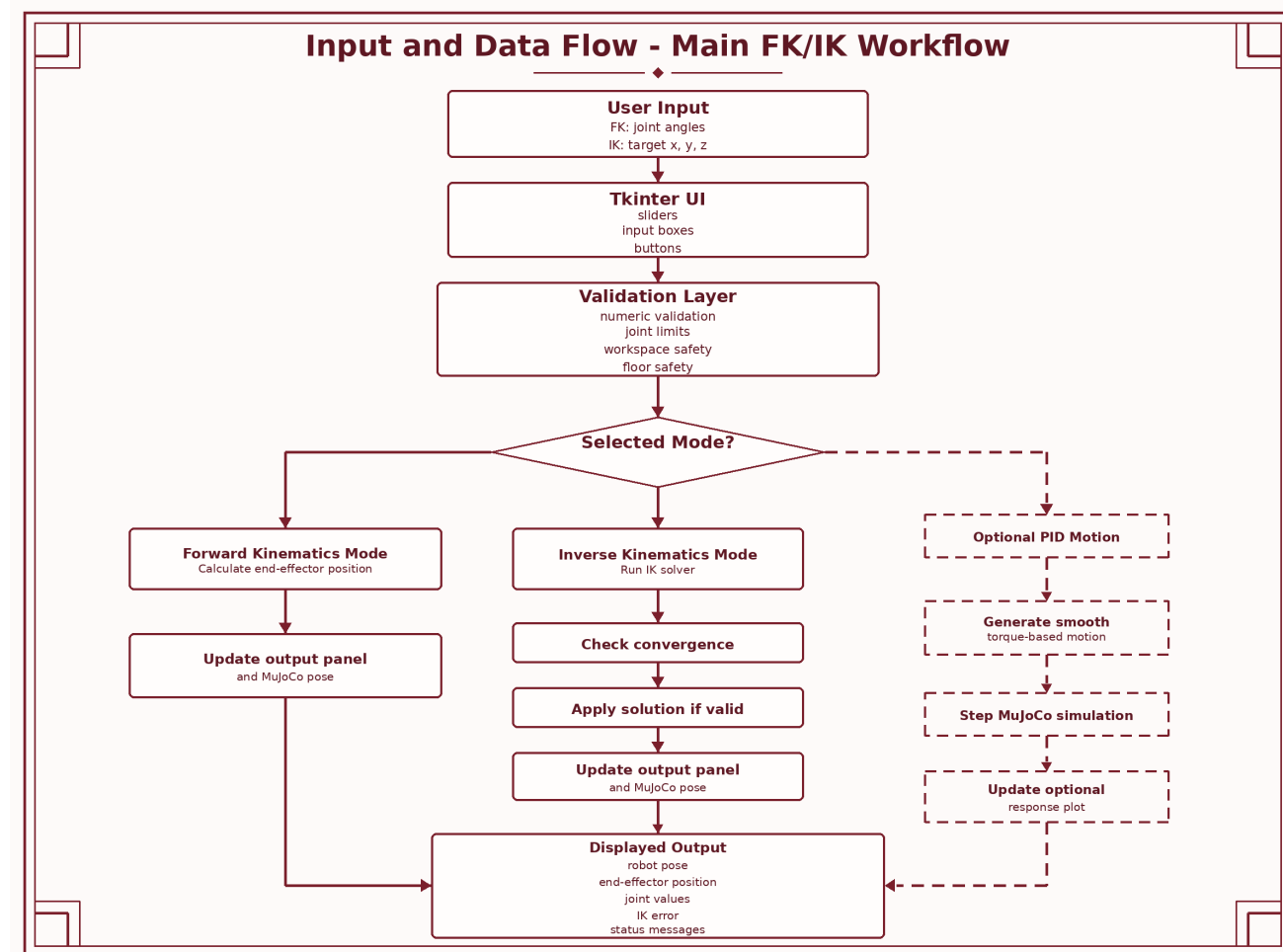


Figure 2: Input and data flow for the main FK/IK workflow, with optional PID motion shown as a secondary branch.

11.4 Non-Functional Requirements

Requirement	Design Response
Performance	FK, IK, optional PID, and UI updates operate on only three joints, so computation is lightweight. MuJoCo runs small physics steps of 0.005 s.
Reliability	Inputs are clamped, invalid typed values are restored, and unsafe poses are rejected.
Security	The main project is a local desktop app and does not collect user data or expose a network service.
Observability	Text output panels show joint values, position, Jacobian, error, torque, and solver messages.
Maintainability	Files are separated by concern: kinematics, control, simulation, UI, and scripts.
Usability	Sliders and numeric input boxes are both provided. Status messages explain what the program is doing.

12. Algorithm and Logic

12.1 Core Algorithm Summary

The program has two primary algorithms: forward kinematics and inverse kinematics. PID control is included as an additional feedback algorithm for smooth motion when that optional mode is enabled.

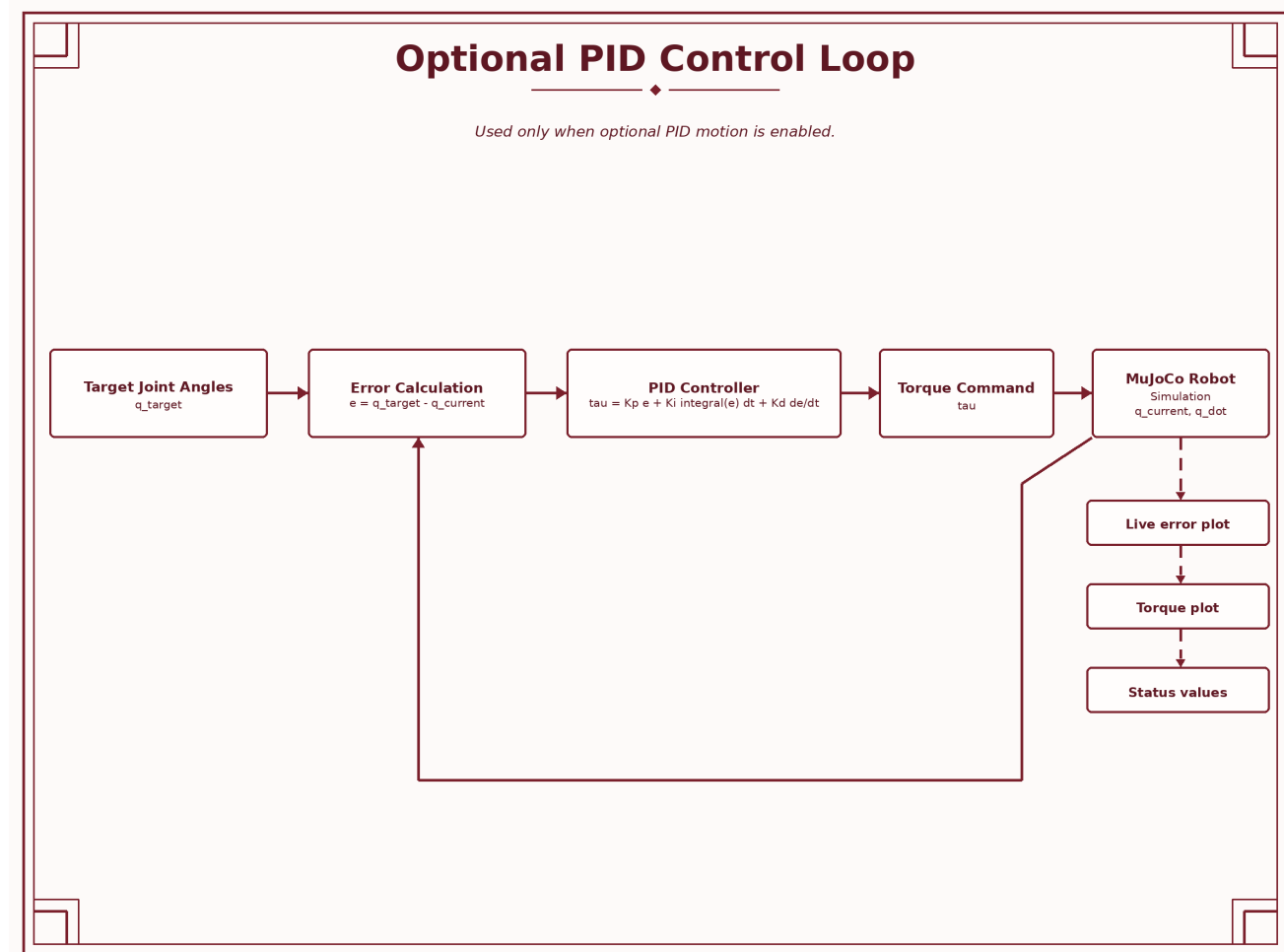


Figure 3: Additional feedback control loop used only when optional PID motion is enabled.

12.2 Flowchart-Style Explanation

1. Start the program using `python main.py` or `./scripts/run.sh`.
2. Create the Tkinter root window and instantiate `KinematicsPidApp`.
3. Load the MuJoCo robot model and initialize robot state.
4. Build tabs for Forward Kinematics, Inverse Kinematics, and the additional PID Control feature.
5. Wait for the user to move a slider, type a value, press a button, or open the viewer.
6. Validate the input against joint limits and safe workspace rules.
7. If FK mode is active, calculate end-effector position and update the output panel.
8. If IK mode is active, move the target marker, solve IK, and optionally apply the solution.
9. If optional PID motion is active, compute torque commands repeatedly and step MuJoCo.
10. Update UI values, plots, status messages, and the MuJoCo viewer.

12.3 Pseudocode

```

1  START
2  load configuration constants
3  create Tkinter window
4  create MuJoCo robot simulation
5  create optional PID controller
6  build FK, IK, and optional PID tabs
7
8  WHILE application is open:
9      IF user changes FK joint slider:
10         read q1, q2, q3
11         validate safe joint position
12         IF optional PID motion is off:
13             set robot joint angles directly
14         ELSE:
15             stage target for optional PID motion
16             update computed position panel
17
18         IF user changes IK target:
19             move red target marker
20             wait for Solve IK button
21
22         IF user clicks Solve IK:
23             compute error = target - current_position
24             use damped least-squares Jacobian updates
25             display joint solution and final error
26
27         IF optional PID is running:
28             compute joint error
29             compute torque using PID equation
30             step MuJoCo simulation
31             update live plot and values
32  END

```

Listing 2: High-level application pseudocode

12.4 Important Decision Points

Decision Point	Program Action
Is the joint pose safe?	If not safe, the program rejects it and restores the previous safe value.
Is optional PID motion enabled?	If enabled, an FK or IK pose can be used as a smooth target. If disabled, the robot pose is applied directly.
Did IK converge?	If yes, the solution can be applied. If no, a clear message explains why.
Is the viewer already open?	The program shows an information message instead of opening a duplicate viewer.
Is a typed value invalid?	The UI restores the previous valid value and reports the issue.

12.5 Input and Output Flow

- **Input:** joint angles in degrees, target position in metres, optional PID gain values, buttons, and checkbox states.
- **Processing:** validation, radians conversion, FK calculation, IK iteration, optional PID torque computation, and MuJoCo stepping.
- **Output:** end-effector position, robot point coordinates, Jacobian matrix, IK solution, error norm, optional PID plot, torque values, status text, and 3D robot simulation.

13. Mathematical Explanation

This section explains the mathematics behind the project. The major concepts are forward kinematics and inverse kinematics. PID control is included as an additional feedback-control concept. Each concept first gives the formula, then explains how to understand it, and finally connects it to the code.

13.1 Concept: Joint Angles and Radians

The UI shows angles in degrees because degrees are easier for students to read. The mathematical functions use radians because NumPy trigonometric functions expect radians.

$$\theta_{rad} = \theta_{deg} \times \frac{\pi}{180}$$

How to Understand This Concept: A full circle is 360° , which is the same as 2π radians. A radian is another unit for angle measurement. Computers often use radians because trigonometric functions such as sine and cosine are naturally defined using radians.

Connection to Code: In the UI action files, degree values from sliders are converted using `np.radians(...)` before they are passed into the kinematics logic and, when used, the optional PID logic.

13.2 Concept: Robot State Vector

The joint angles are represented as a vector:

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

where q_1 is base yaw, q_2 is shoulder pitch, and q_3 is elbow pitch.

The end-effector position is represented as:

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

How to Understand This Concept: A vector is a compact way of storing related numbers together. Instead of handling three separate variables repeatedly, the code can handle one array containing all three joint angles or all three coordinates.

Connection to Code: The project uses NumPy arrays for $\mathbf{q}$ and $\mathbf{p}$. Functions such as `forward_kinematics()` return a three-value array $[x, y, z]$.

13.3 Concept: Forward Kinematics

Forward kinematics calculates the end-effector position from known joint angles. The link lengths are:

$$L_1 = 0.45 \text{ m}, \quad L_2 = 0.35 \text{ m}, \quad L_3 = 0.25 \text{ m}$$

The base height is:

$$h = 0.18 \text{ m}$$

The project combines L_2 and L_3 because they share the same elbow angle direction:

$$L_{23} = L_2 + L_3$$

The radial reach from the vertical axis is:

$$r = L_1 \cos(q_2) + (L_2 + L_3) \cos(q_2 + q_3)$$

The Cartesian position is:

$$x = r \cos(q_1)$$

$$y = r \sin(q_1)$$

$$z = h + L_1 \sin(q_2) + (L_2 + L_3) \sin(q_2 + q_3)$$

How to Understand This Concept: Imagine looking at the arm from the side. The shoulder and elbow decide how far the arm reaches outward and how high it rises. Then the base yaw rotates that side-view reach around the vertical axis to produce the final x and y position.

Connection to Code: The file `kinematics/forward.py` implements these exact equations in the function `forward_kinematics()`.

```

1 def forward_kinematics(joint_angles, link_lengths=LINK_LENGTHS, base_height=BASE_HEIGHT):
2     q1, q2, q3 = as_three_values(joint_angles, "joint_angles")
3     l1, l2, l3 = as_three_values(link_lengths, "link_lengths")
4
5     forearm_and_tool = l2 + l3
6     elbow_angle = q2 + q3
7
8     radial_reach = l1 * np.cos(q2) + forearm_and_tool * np.cos(elbow_angle)
9     x = radial_reach * np.cos(q1)
10    y = radial_reach * np.sin(q1)
11    z = base_height + l1 * np.sin(q2) + forearm_and_tool * np.sin(elbow_angle)
12
13    return np.array([x, y, z], dtype=float)

```

Listing 3: `kinematics/forward.py` - forward kinematics calculation

13.4 Concept: Link Position Geometry

The elbow point can be understood using the direction of yaw and height:

$$\mathbf{p}_{elbow} = \begin{bmatrix} 0 \\ 0 \\ h \end{bmatrix} + L_1 \begin{bmatrix} \cos(q_1) \cos(q_2) \\ \sin(q_1) \cos(q_2) \\ \sin(q_2) \end{bmatrix}$$

How to Understand This Concept: A link has a length and a direction. The direction is split into horizontal movement and vertical movement. The horizontal movement is then rotated around the base by q_1 .

Connection to Code: The function `joint_positions()` calculates floor base, shoulder, elbow, wrist/tool point, and end-effector coordinates. The wrist/tool point is an intermediate point in the model and is not an additional controlled degree of freedom. These points are used for output and safety checks.

Kinematic Geometry for $q = [25 \text{ deg}, 35 \text{ deg}, -45 \text{ deg}]$

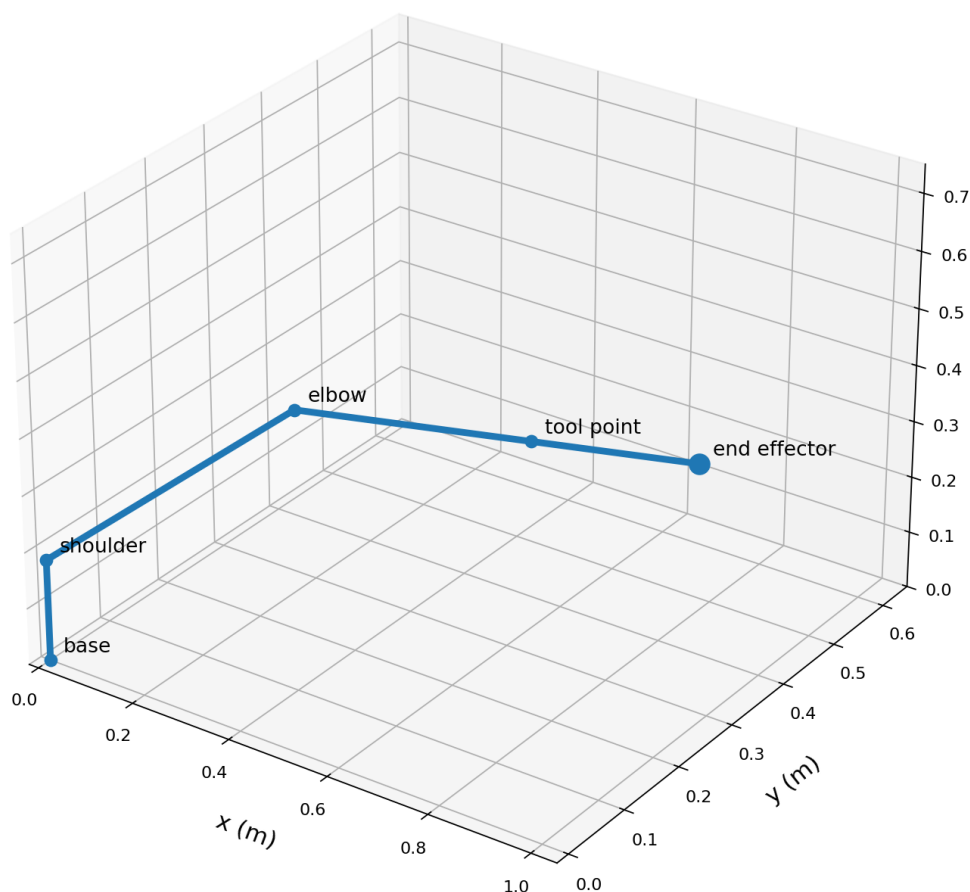


Figure 4: Kinematic geometry produced from the project equations. The wrist/tool point is intermediate and is not an additional actuated joint.

13.5 Concept: Jacobian Matrix

The Jacobian matrix describes how small changes in joint angles change the end-effector position:

$$J(\mathbf{q}) = \frac{\partial \mathbf{p}}{\partial \mathbf{q}} = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} & \frac{\partial x}{\partial q_3} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} & \frac{\partial y}{\partial q_3} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} & \frac{\partial z}{\partial q_3} \end{bmatrix}$$

For small changes:

$$\Delta \mathbf{p} \approx J(\mathbf{q}) \Delta \mathbf{q}$$

How to Understand This Concept: The Jacobian is like a sensitivity table. It tells the program which joint is most responsible for moving the hand in each direction. If changing one joint changes x a lot, the corresponding Jacobian value is large.

Connection to Code: The function `jacobian()` in `kinematics/forward.py` returns the 3×3 matrix used by the IK solver.

13.6 Concept: Inverse Kinematics Error

Inverse kinematics starts from the difference between the desired target and the current position:

$$\mathbf{e} = \mathbf{p}_{target} - \mathbf{p}_{current}$$

The solver wants $\|\mathbf{e}\|$ to become very small:

$$\|\mathbf{e}\| < 10^{-4}$$

How to Understand This Concept: Error is the remaining distance from where the end effector is to where it should be. If the error is zero, the robot has reached the target exactly.

Connection to Code: The helper function computes target position minus current position. The IK result stores the final error vector and its norm.

13.7 Concept: Damped Least-Squares Inverse Kinematics

The project uses damped least-squares IK:

$$\Delta \mathbf{q} = \alpha \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \mathbf{e}$$

where α is the step size and λ is the damping value.

How to Understand This Concept: The solver repeatedly asks: “Which small joint movement will reduce the position error?” The Jacobian gives the direction. The damping term prevents the calculation from becoming unstable when the arm is near difficult or stretched positions.

Connection to Code: The inverse kinematics function forms a damped stability matrix and solves a small linear system to find the joint step.

```

1 robot_jacobian = jacobian(joint_angles, link_lengths)
2 stable_matrix = robot_jacobian @ robot_jacobian.T + (damping**2) * identity
3 joint_step = step_size * robot_jacobian.T @ np.linalg.solve(
4     stable_matrix,
5     final_error,
6 )
7
8 step_norm = float(np.linalg.norm(joint_step))
9 if step_norm > max_step:
10     joint_step *= max_step / step_norm
11
12 joint_angles = clamp_to_joint_limits(joint_angles + joint_step)

```

Listing 4: kinematics/inverse.py - damped least-squares IK step

13.8 Concept: Workspace and Safety Limits

The horizontal distance from the base axis to the target point is:

$$r = \sqrt{x^2 + y^2}$$

The distance from shoulder to target is:

$$d = \sqrt{r^2 + (z - h)^2}$$

where $r = \sqrt{x^2 + y^2}$ is the horizontal distance from the base axis to the target point. A target is physically reachable only if:

$$|L_1 - (L_2 + L_3)| \leq d \leq L_1 + L_2 + L_3$$

The project also checks that moving robot points remain above the floor:

$$z_{min} \geq 0.02 \text{ m}$$

How to Understand This Concept: A robot arm cannot reach infinitely far. It also should not pass through the floor. Workspace rules describe the safe region where the robot can move naturally.

Connection to Code: The geometric IK candidate logic, safety-message function, and safe-position check work together to check reachability, joint limits, and floor safety.

13.9 Concept: PID Control

PID control is an additional feature in this project. It uses proportional, integral, and derivative terms to show smooth feedback-based movement after a target pose is selected:

$$\tau = K_p e + K_i \int e \, dt + K_d \frac{de}{dt}$$

In discrete code, the integral is updated as:

$$I_k = I_{k-1} + e_k \Delta t$$

The derivative term is related to velocity:

$$D_k \approx -\dot{q}_k$$

How to Understand This Concept: The proportional term reacts to current error. The integral term reacts to accumulated error over time. The derivative term reacts to motion speed and helps reduce overshoot. Together, they make the robot move toward the target smoothly.

Connection to Code: The `PIDController.compute()` method calculates error, updates integral error, estimates derivative from velocity, computes torque, and clips torque to a safe limit.

```

1 def compute(self, target_angles, current_angles, current_velocities, dt):
2     if dt <= 0.0:
3         raise ValueError("dt must be positive.")
4
5     error = wrap_angles(target_angles - current_angles)
6     self.integral_error += error * dt
7     self.integral_error = np.clip(
8         self.integral_error,
9         -self.integral_limit,
10        self.integral_limit,
11    )
12
13    derivative = -current_velocities
14    torque = self.kp * error + self.ki * self.integral_error + self.kd * derivative
15    return np.clip(torque, -self.torque_limit, self.torque_limit)

```

Listing 5: control/pid.py - PID torque calculation

13.10 Concept: Simulation Time Step

MuJoCo steps the simulation using a fixed timestep:

$$\Delta t = 0.005 \text{ s}$$

The project performs three simulation steps per UI frame:

$$T_{frame} = 3\Delta t = 0.015 \text{ s}$$

How to Understand This Concept: A simulation does not move continuously in one infinite calculation. It moves in very small time steps. Smaller steps usually make the simulation more stable and realistic.

Connection to Code: The simulation timestep and steps-per-frame values are stored in the configuration file. The PID stepping method repeatedly calls the MuJoCo step function.

13.11 Concept: Algorithmic Complexity

For this fixed 3-DOF implementation, forward kinematics and PID control run in constant time, $O(1)$, because the number of joints is always three:

$$T_{FK} = O(1), \quad S_{FK} = O(1)$$

$$T_{PID} = O(1), \quad S_{PID} = O(1)$$

For a generalized n -joint robot, forward kinematics and PID control can be described as:

$$T_{FK}(n) = O(n), \quad T_{PID}(n) = O(n)$$

For this fixed 3-DOF robot, each IK iteration solves a small fixed-size 3×3 system, so the practical complexity is:

$$T_{IK}(k) = O(k), \quad S_{IK} = O(1)$$

where k is the number of solver iterations. In a generalized robot model with larger matrices, the matrix solve cost may grow with the matrix dimension.

How to Understand This Concept: Complexity describes how much work an algorithm does as the problem size grows. In this project, the robot always has three joints, so each FK or PID calculation has a fixed amount of work. IK repeats a fixed-size calculation several times, so its practical cost mainly depends on the number of iterations.

Connection to Code: The application remains responsive because the math modules operate on small NumPy arrays. The IK solver has a configured maximum iteration count, and the UI update interval prevents the interface from becoming overloaded.

14. UI Explanation

14.1 Main UI Layout

The UI is built using Tkinter and `ttk`. The main window contains a top bar, a tabbed notebook, and a status bar. The top bar includes the project title, an optional **Use PID Motion** checkbox, **Reset Robot** button, and **Open MuJoCo Viewer** button.

```

1  ttk.Label(title_area, text="3D 3-DOF Robot Arm", style="Title.TLabel").pack(anchor=tk.W)
2  ttk.Label(
3      title_area,
4      text="Forward kinematics - Inverse kinematics - PID control",
5      style="Subtitle.TLabel",
6  ).pack(anchor=tk.W, pady=(1, 0))
7
8  ttk.Button(
9      top_bar,
10     text="Open MuJoCo Viewer",
11     style="Accent.TButton",
12     command=app.open_mujoco_viewer,
13 ).pack(side=tk.RIGHT, padx=(6, 0))

```

Listing 6: ui/layout.py - top-level layout components

14.2 Forward Kinematics Tab

The Forward Kinematics tab contains sliders and numeric input boxes for q_1 , q_2 , and q_3 . When direct mode is active, changing the sliders immediately moves the robot and updates the output.

```

1  q_controls = [
2      add_slider(input_panel, "q1 yaw (deg)", -180, 180, 0, 0, 0, "{:.1f}", ...),
3      add_slider(input_panel, "q2 shoulder (deg)", 0, 110, 0, 1, 0, "{:.1f}", ...),
4      add_slider(input_panel, "q3 elbow (deg)", -135, 0, 0, 2, 0, "{:.1f}", ...),
5  ]
6
7  add_latex_equation_box(
8      math_panel,
9      equations=[
10         "r = L1*cos(q2) + (L2 + L3)*cos(q2 + q3)",
11         "x = r*cos(q1)",
12         "y = r*sin(q1)",
13         "z = h + L1*sin(q2) + (L2 + L3)*sin(q2 + q3)",
14     ],
15 )

```

Listing 7: ui/forward_tab.py - FK controls and equations

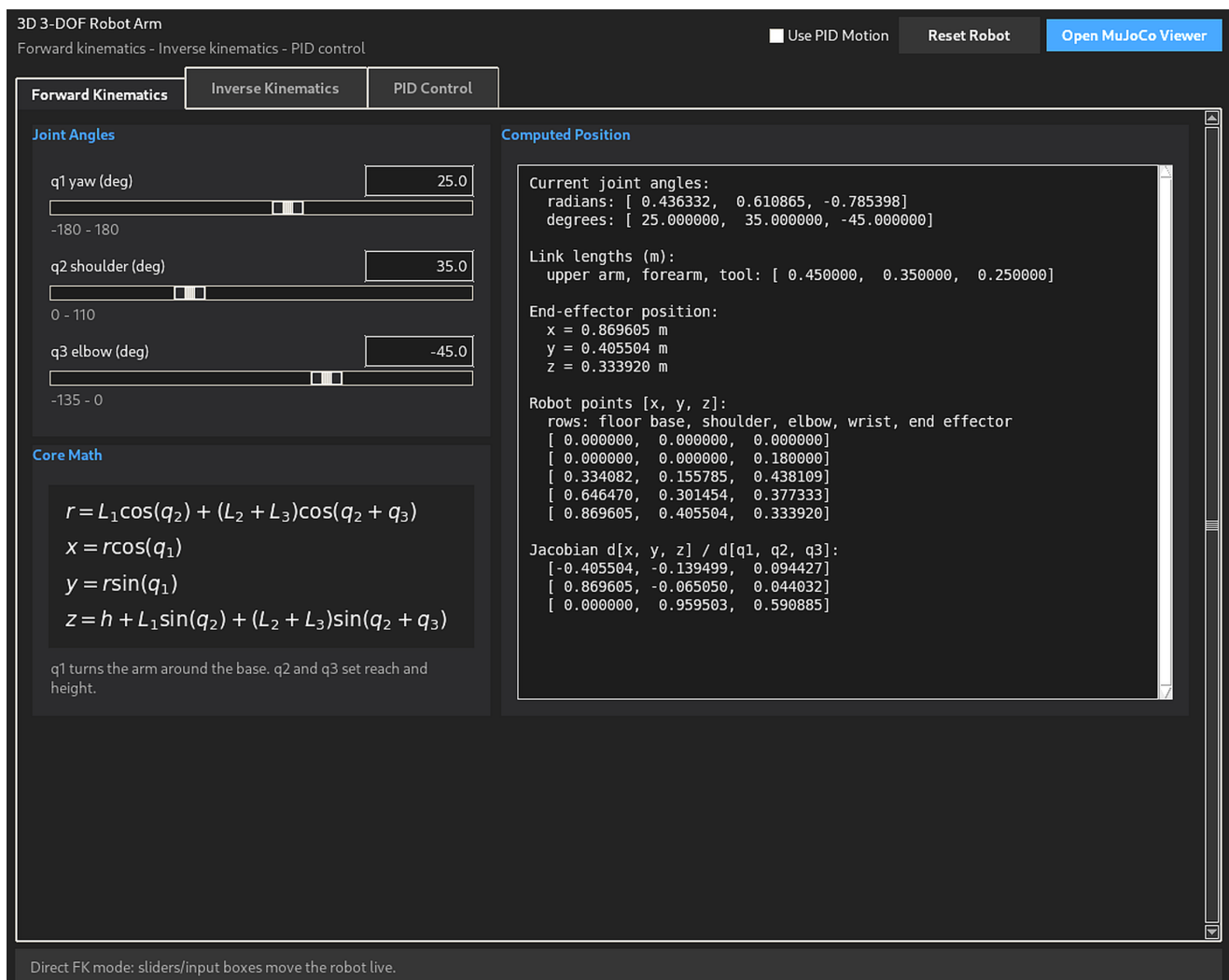


Figure 5: Actual Tkinter Forward Kinematics tab showing joint inputs, equations, end-effector output, joint positions, and Jacobian values.

14.3 Inverse Kinematics Tab

The Inverse Kinematics tab contains target sliders for x , y , and z . Moving the sliders changes the target marker. Pressing **Solve IK** calculates the joint angles. Pressing **Apply Solution** moves the robot to the solution if the solver converged.

```
1 x_control = add_slider(target_panel, "target x (m)", -IK_TARGET_XY_LIMIT, IK_TARGET_XY_LIMIT,
2   DEFAULT_TARGET_POSITION[0], ...)
3 y_control = add_slider(target_panel, "target y (m)", -IK_TARGET_XY_LIMIT, IK_TARGET_XY_LIMIT,
4   DEFAULT_TARGET_POSITION[1], ...)
5 z_control = add_slider(target_panel, "target z (m)", MIN_SAFE_Z, IK_TARGET_Z_MAX,
6   DEFAULT_TARGET_POSITION[2], ...)
7
8 ttk.Button(button_row, text="Solve IK", command=on_solve, style="Accent.TButton").pack(side=tk.LEFT)
9 ttk.Button(button_row, text="Apply Solution", command=on_apply_solution).pack(side=tk.LEFT,
10   padx=(8, 0))
```

Listing 8: ui/inverse_tab.py - IK target controls and solver buttons

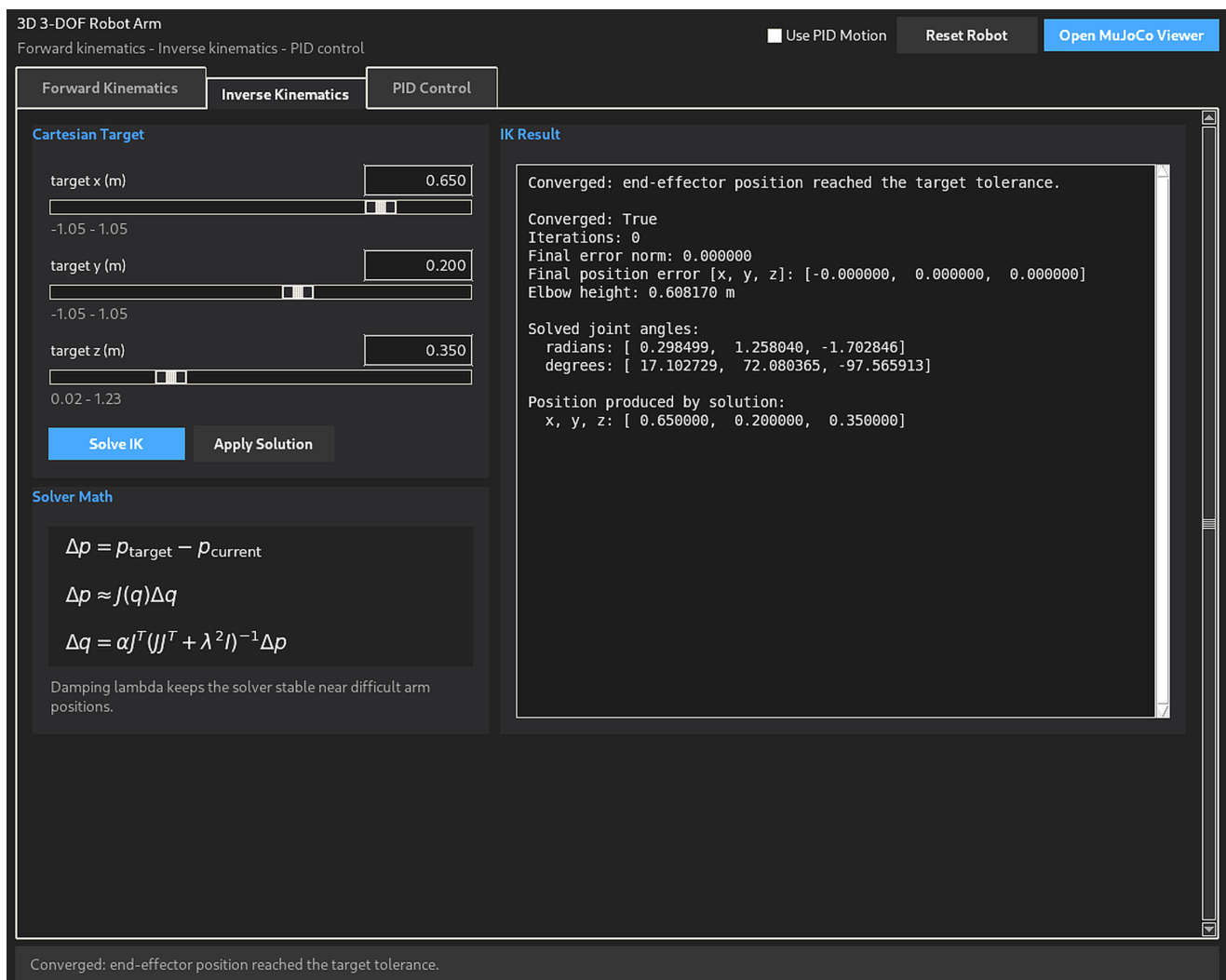


Figure 6: Actual Tkinter Inverse Kinematics tab showing a converged target solution and final error information.

14.4 PID Control Tab

The PID tab is an additional feature. It contains target joint sliders, per-joint PID gain sliders, buttons, live value display, and a Matplotlib response plot. This tab shows how joint error and torque evolve over time when optional smooth motion is enabled.

```
1 q_controls = [
2     add_slider(target_panel, "target yaw q1 (deg)", -180, 180, DEFAULT_PID_TARGET_DEGREES[0], ...),
3     add_slider(target_panel, "target shoulder q2 (deg)", 0, 110, DEFAULT_PID_TARGET_DEGREES[1],
4         ...),
5     add_slider(target_panel, "target elbow q3 (deg)", -135, 0, DEFAULT_PID_TARGET_DEGREES[2], ...),
6 ]
7 (error_line,) = axis.plot([], [], label="joint error norm", color=UI_COLORS["accent"])
8 (torque_line,) = axis.plot([], [], label="torque norm", color=UI_COLORS["accent_secondary"])
```

Listing 9: ui/pid_tab.py - PID target, gain, and plot UI

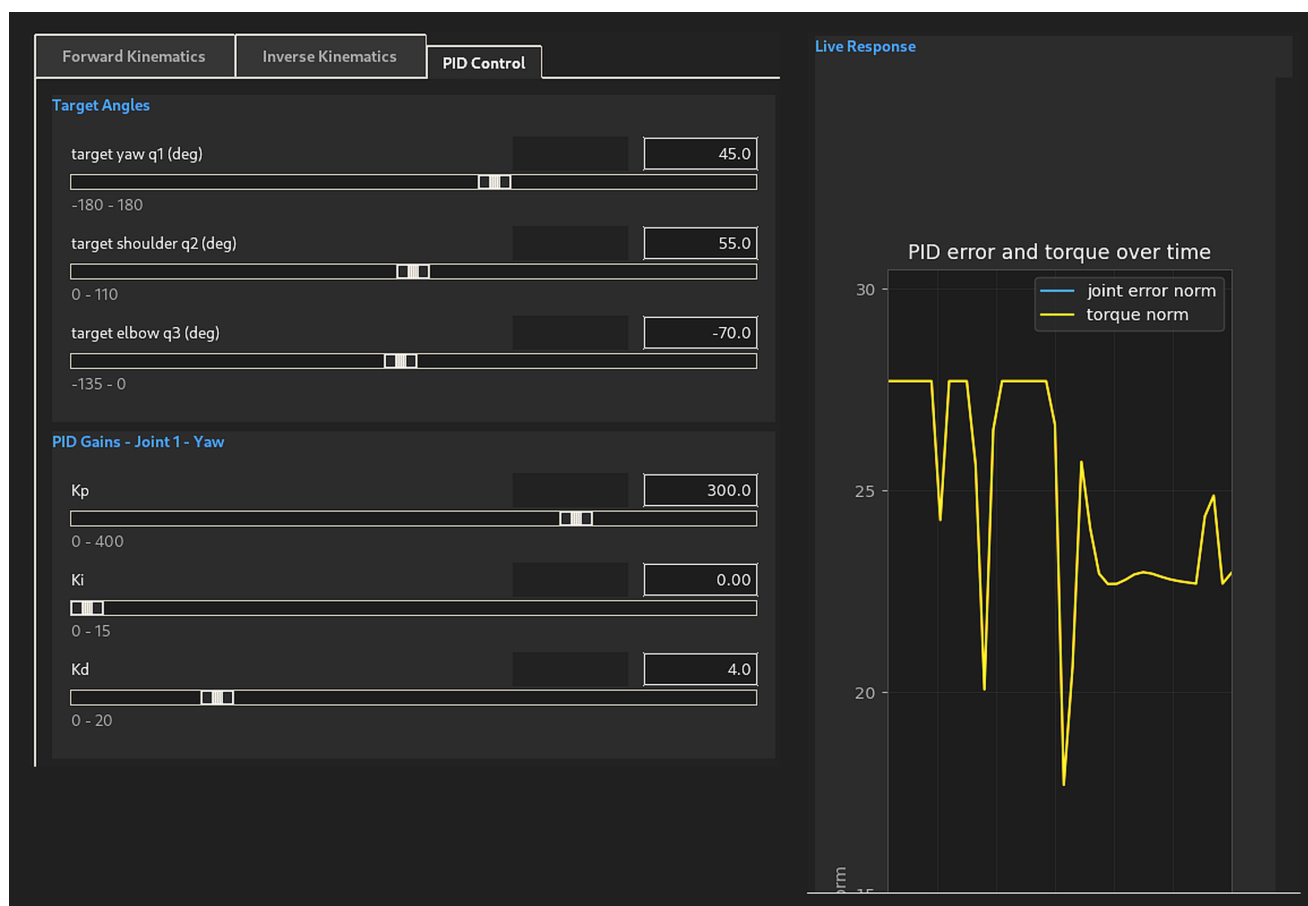


Figure 7: Actual Tkinter PID Control tab crop showing the optional target angles, PID gains, and live response plot more clearly.

14.5 How UI Events Connect to Backend Logic

When the user interacts with sliders or buttons, the UI calls action functions. These action functions read values, validate them, run math functions, update simulation state, and write output back to the UI.

```

1 def on_slider_changed(app):
2     joint_angles = np.radians(app._read_slider_values(app.fk_q_controls))
3     if not accept_slider_angles(app, joint_angles):
4         return
5
6     if app.pid_motion_enabled_var.get():
7         app.status_var.set("FK target staged. Click Apply FK Target to start PID motion.")
8         return
9
10    app._set_joint_angles(joint_angles)
11    update_output(app)
12    app.status_var.set("Direct FK mode: sliders/input boxes move the robot live.")

```

Listing 10: ui/fk_actions.py - FK slider event logic

15. Main Logic Code Explanation

15.1 Application Entry Point

The `main.py` file starts the program and handles missing dependencies. If Tkinter or another dependency is missing, the program prints a helpful installation message instead of failing silently.

```

1 def main() -> int:
2     """Start the Tkinter application."""
3
4     try:
5         import tkinter as tk
6         from ui.app import KinematicsPidApp
7     except ImportError as error:
8         print("Could not start the app because a dependency is missing.")
9         print(f"Missing detail: {error}")
10        print("Install dependencies with:")
11        print("  sudo apt install python3-tk")
12        print("  python -m pip install -r requirements.txt")
13        return 1
14
15    root = tk.Tk()
16    app = KinematicsPidApp(root)
17    app.run()
18    return 0

```

Listing 11: `main.py` - application entry point

15.2 Configuration Values

The `config.py` file centralizes all constants. This avoids scattering link lengths, joint limits, optional PID gains, and simulation settings throughout the code.

```

1 LINK_LENGTHS = np.array([0.45, 0.35, 0.25], dtype=float)
2 BASE_HEIGHT = 0.18
3
4 JOINT_LIMITS_DEGREES = np.array(
5     [[-180.0, 180.0], [0.0, 110.0], [-135.0, 0.0]],
6     dtype=float,
7 )
8 MIN_SAFE_Z = 0.02
9
10 SIMULATION_TIMESTEP = 0.005
11 SIMULATION_STEPS_PER_FRAME = 3
12 TORQUE_LIMIT = 16.0
13 DEFAULT_PID_GAINS = {
14     "joint1": (300.0, 0.0, 4.0),
15     "joint2": (320.0, 11.5, 4.7),
16     "joint3": (283.3, 0.0, 4.0),
17 }

```

Listing 12: `config.py` - important constants

15.3 Safety Validation

Safety validation protects the demo from unnatural and below-floor poses.

```

1 def safe_joint_message(joint_angles, link_lengths=LINK_LENGTHS, base_height=BASE_HEIGHT,
2     min_safe_z=MIN_SAFE_Z):
3     joint_angles = wrap_angles(as_three_values(joint_angles, "joint_angles"))
4     q1, q2, q3 = joint_angles
5     lower_limits = JOINT_LIMITS_RADIANS[:, 0]
6     upper_limits = JOINT_LIMITS_RADIANS[:, 1]
7
8     if q2 < lower_limits[1]:
9         return "Blocked: shoulder-down poses are not allowed."

```

```

9  if q3 > upper_limits[2]:
10     return "Blocked: positive/back-folding elbow poses are not allowed."
11
12     moving_points = joint_positions(joint_angles, link_lengths, base_height)[1:]
13     minimum_point_z = float(np.min(moving_points[:, 2]))
14     if minimum_point_z < min_safe_z:
15         return "Blocked: the arm would go below the floor."
16
17     return ""

```

Listing 13: kinematics/inverse.py - safe joint pose validation

15.4 Inverse Kinematics Result Object

The IK solver returns an `IKResult` object. This makes the result easy to display because it contains both the solution and diagnostic information.

```

1  @dataclass
2  class IKResult:
3      """Result returned by the inverse kinematics solver."""
4
5      joint_angles: np.ndarray
6      converged: bool
7      iterations: int
8      final_error: np.ndarray
9      error_norm: float
10     message: str

```

Listing 14: kinematics/inverse.py - IKResult data structure

15.5 MuJoCo Model Creation

The project creates the MuJoCo robot model as an MJCF XML string. It defines the base, shoulder, elbow, tool link, target marker, and three motors.

```

1  def create_mjcf() -> str:
2      """Return the MJCF XML model as a string."""
3
4      l1, l2, l3 = LINK_LENGTHS
5      target_x, target_y, target_z = DEFAULT_TARGET_POSITION
6
7      return f"""
8  <mujoco model="three_dof_xyz_arm">
9      <compiler angle="radian" coordinate="local"/>
10     <option timestep="{SIMULATION_TIMESTEP}" gravity="0 0 0"/>
11
12     <worldbody>
13         <body name="target" mocap="true" pos="{target_x} {target_y} {target_z}">
14             <geom name="target_marker" type="sphere" size="0.035" rgba="0.95 0.20 0.20 0.70"/>
15         </body>
16         ... robot bodies and hinge joints ...
17     </worldbody>
18
19     <actuator>
20         <motor name="motor1" joint="joint1" ctrlrange="-{TORQUE_LIMIT} {TORQUE_LIMIT}"/>
21         <motor name="motor2" joint="joint2" ctrlrange="-{TORQUE_LIMIT} {TORQUE_LIMIT}"/>
22         <motor name="motor3" joint="joint3" ctrlrange="-{TORQUE_LIMIT} {TORQUE_LIMIT}"/>
23     </actuator>
24 </mujoco>
25 """ .strip()

```

Listing 15: simulation/mujoco_sim.py - MuJoCo model and actuators

15.6 PID Simulation Step

For the additional PID feature, the program enables gravity, computes torque, writes motor commands, and steps MuJoCo.

```

1 def step_pid_control(self, controller: PIDController, target_angles: np.ndarray) -> PidStepSample:
2     """Advance MuJoCo a few small steps using PID torque commands."""
3
4     with self._viewer_lock():
5         self.set_gravity_enabled(True)
6         for _ in range(SIMULATION_STEPS_PER_FRAME):
7             torque = controller.compute(
8                 target_angles,
9                 self.data.qpos[:3],
10                self.data.qvel[:3],
11                SIMULATION_TIMESTEP,
12            )
13            self.data.ctrl[:3] = torque
14            mujoco.mj_step(self.model, self.data)
15
16        error = wrap_angles(target_angles - self.data.qpos[:3])
17        return PidStepSample(
18            elapsed_time=SIMULATION_TIMESTEP * SIMULATION_STEPS_PER_FRAME,
19            error_norm=float(np.linalg.norm(error)),
20            torque_norm=float(np.linalg.norm(self.data.ctrl[:3])),
21        )

```

Listing 16: simulation/mujoco_sim.py - PID simulation stepping

16. Demo / Program Run-through

This section explains how a teacher or examiner can run and understand the project from start to finish.

16.1 Setup Commands

```

1 # On Ubuntu, install Tkinter first if needed
2 sudo apt install python3-tk
3
4 # Set up the project virtual environment
5 ./scripts/setup.sh
6
7 # Run the project
8 ./scripts/run.sh
9
10 # Manual fallback
11 python3 -m venv .env
12 .env/bin/python -m pip install -r requirements.txt
13 .env/bin/python main.py

```

Listing 17: Setup and run commands

16.2 Step-by-Step Walkthrough

1. **Launch the program.** Run `./scripts/run.sh` or `python main.py`.
2. **Open the Tkinter interface.** The main window opens with Forward Kinematics, Inverse Kinematics, and an additional PID Control tab.
3. **Open the MuJoCo viewer if needed.** Click **Open MuJoCo Viewer** to see the 3D robot arm in a separate window.
4. **Use the Forward Kinematics tab.** Select the Forward Kinematics tab.
5. **Move the joint sliders.** Adjust yaw, shoulder, and elbow values.
6. **Observe FK output.** The computed end-effector position, robot point coordinates, and Jacobian update in the output panel.
7. **Use the Inverse Kinematics tab.** Select the Inverse Kinematics tab.
8. **Adjust a target position.** Enter or slide the desired target values for x , y , and z .
9. **Solve IK.** Click **Solve IK** to calculate the joint angles required for the target position.
10. **Apply the IK solution.** Click **Apply Solution** if the solver reports convergence.
11. **Observe the final result.** Check the updated robot pose, solved joint angles, final error vector, and final error norm.

The PID tab remains available as an extra feature for showing smooth joint-space movement, but it is not required for the main FK and IK demonstration.

16.3 MuJoCo Simulation Output

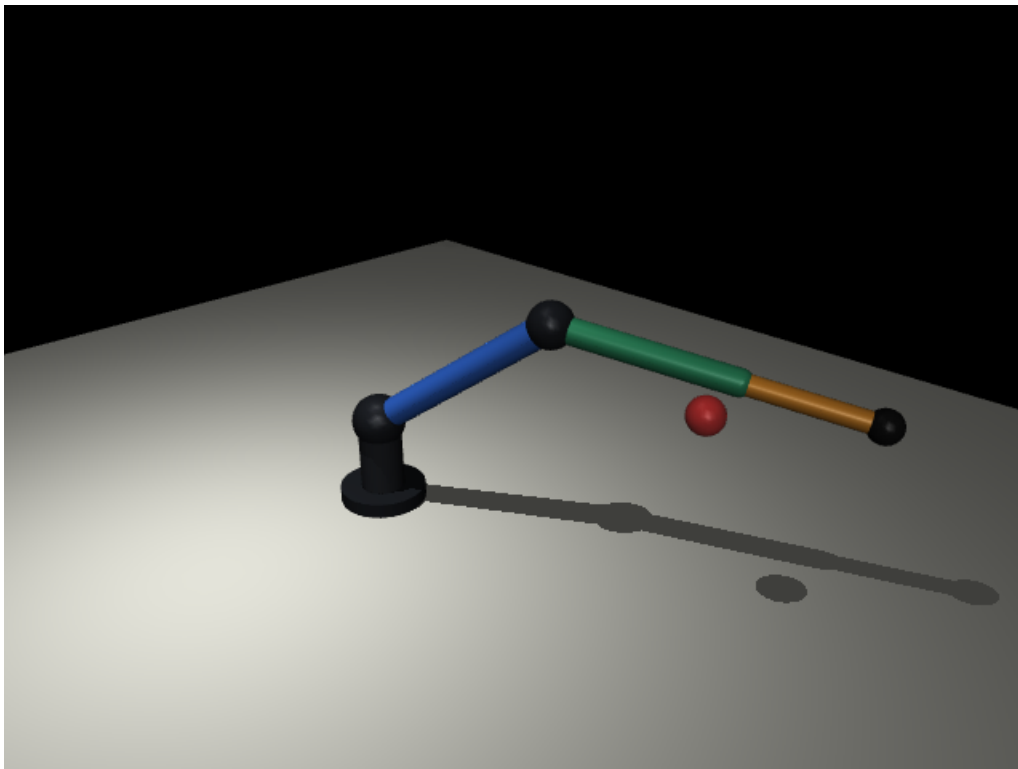


Figure 8: Actual MuJoCo-rendered robot arm pose with target marker. The colored links represent the base, upper arm, forearm, and tool.

16.4 Backend Output During Demo

The backend computes the same values that are displayed in the UI. The following screenshot was generated by running the project math modules and printing the FK, Jacobian, and IK result.

```
$ python main.py
Ready. Open the MuJoCo viewer to see the 3D arm.

Backend math check from project modules:
FK q(deg)=[25, 35, -45] -> p=[0.869605 0.405504 0.33392 ]
Jacobian shape=(3, 3)
IK target=[0.65,0.20,0.35] converged=True, iterations=0, error_norm=1.110223e-16
IK solution deg=[ 17.103  72.08 -97.566]
```

Figure 9: Backend output generated from the actual project functions for FK, Jacobian, and IK.

16.5 What Happens Internally During the Demo

When the user changes a value, the UI converts the input into a NumPy array. Joint angles are converted from degrees to radians. The safety function checks the pose. For the main FK and IK demonstration, safe values are written directly to the MuJoCo state and the UI output is refreshed. If the optional PID feature is later enabled, the selected target is stored and the controller computes torques over many small simulation steps.

17. Results and Discussion

17.1 Final Output of the Project

The project successfully demonstrates the main FK and IK workflow: input through the UI, mathematical processing, safe validation, simulation, and visible output. The FK tab shows joint angles, link lengths, end-effector position, joint coordinates, and the Jacobian. The IK tab shows whether the target was reached, the number of iterations, final error, and solved joint angles. The additional PID tab shows live target, current angles, error, torque, gravity state, and the response graph for smooth-motion experiments.

17.2 Objective Achievement

Objective	Evidence	Status
Forward kinematics	FK equations are implemented and shown in the UI output.	Achieved
Inverse kinematics	Damped least-squares solver reaches valid targets and reports convergence.	Achieved
Additional PID feature	Controller produces torque commands and records response history for smooth-motion demonstration.	Achieved
MuJoCo visualization	The robot model renders with joints, links, tool, and target marker.	Achieved
Usable interface	Tkinter tabs provide sliders, entries, buttons, outputs, and status messages.	Achieved

17.3 Strengths of the Implementation

- Clear modular structure with separate folders for math, control, simulation, and UI.
- Student-friendly interface with direct visual feedback.
- Real robotics concepts implemented in executable code.
- Safety limits prevent unrealistic robot poses.
- Optional PID history and live plot make feedback control visible without changing the main FK/IK focus.
- MuJoCo model links the mathematical robot to a physical-style simulation.

17.4 Limitations of the Current Version

- The arm has only three degrees of freedom and controls position, not full end-effector orientation.
- PID gains are manually configured and may need tuning for different robot sizes.
- The UI is desktop-based and not available as a browser or mobile application.
- The project does not currently save sessions, logs, or user experiments to a database.
- MuJoCo viewer requires working graphics support, which may not be available on every system.

17.5 Possible Improvements

The project can be improved by adding a database for experiment logs, adding more robot joints, adding a trajectory planner, providing exportable plots, improving collision detection, and creating a web dashboard for remote demonstrations.

18. Future Applications

Future Application	Explanation
Robotics education	Students can use the project to learn forward kinematics, inverse kinematics, vectors, matrices, optional feedback control, and simulation.
Industrial robot prototyping	The same architecture can be extended to test pick-and-place motions before using physical hardware.
Trajectory planning	Future versions can generate smooth paths instead of only point-to-point motion.
Database integration	SQLite or cloud storage can save target positions, controller gains, run results, and response data.
Cloud deployment	A web version could show plots and simulation output through a browser.
Mobile app version	A mobile interface could allow remote control or classroom demonstrations.
AI/ML integration	A learning model could tune PID gains automatically or predict reachable poses.
Security improvements	User input validation and sandboxing can be expanded if the project becomes networked.
Automation	The project can become a base for scripted robotic tasks such as sorting or object positioning.
Control systems extension	More advanced controllers such as LQR, MPC, or computed-torque control can be added after the FK and IK foundation is understood.

19. Conclusion

This project demonstrates a complete and practical computer science application based on robotics. Its main focus is forward kinematics and inverse kinematics for a 3D 3-DOF robot arm. It also uses Python programming, Tkinter UI design, NumPy mathematics, Matplotlib visualization, MuJoCo simulation, and an additional PID control feature.

The main learning from the project is that robotics software is not only about formulas or only about graphics. A working robotic demonstration needs a clear architecture: user inputs must be validated, FK and IK logic must be correct, and the simulation must show the result clearly. Optional feedback control can then be added to improve motion smoothness.

The project also shows how Class XII-level mathematics such as trigonometry, vectors, matrices, and iterative algorithms can be applied to real engineering problems. It is useful because it turns abstract ideas into a working program that can be demonstrated, studied, and extended.

Final Statement

The 3D 3-DOF Robot Arm project successfully meets its objective of presenting forward kinematics, inverse kinematics, and simulation in an interactive and examiner-friendly form, with PID control included as an additional smooth-motion feature.

20. Appendix

20.1 Setup Notes

```

1 # Clone repository
2 git clone https://github.com/joeljebitto-dev/School_cs_project_12th.git
3 cd School_cs_project_12th
4
5 # Create environment and install packages
6 ./scripts/setup.sh
7
8 # Run application
9 ./scripts/run.sh
10
11 # Optional repository checks
12 .venv/bin/python -m pytest

```

Listing 18: Useful setup commands

20.2 Key Constants from Configuration

Constant	Meaning
$L_1 = 0.45\text{ m}, L_2 = 0.35\text{ m}, L_3 = 0.25\text{ m}$	Robot link lengths.
$h = 0.18\text{ m}$	Shoulder/base height above the floor.
$q_1 \in [-180^\circ, 180^\circ]$	Base yaw range.
$q_2 \in [0^\circ, 110^\circ]$	Shoulder pitch range.
$q_3 \in [-135^\circ, 0^\circ]$	Elbow pitch range.
$z_{min} = 0.02\text{ m}$	Minimum safe height above floor.
$\Delta t = 0.005\text{ s}$	MuJoCo simulation timestep.
$ \tau \leq 16$	Torque command limit.

20.3 Additional Code Snippet: Setup Script

```

1 def main() -> int:
2     if not REQUIREMENTS_FILE.exists():
3         print("Could not find requirements.txt.")
4         return 1
5
6     if not VENV_DIR.exists():
7         print("Creating virtual environment in .venv ...")
8         venv.EnvBuilder(with_pip=True).create(VENV_DIR)
9
10    python_path = venv_python_path()
11    run_command([str(python_path), "-m", "pip", "install", "--upgrade", "pip"])
12    run_command([str(python_path), "-m", "pip", "install", "-r", str(REQUIREMENTS_FILE)])

```

Listing 19: scripts/setup_project.py - environment setup pattern

20.4 Additional Diagram: Control Data Flow

The following diagram summarizes how user commands are validated and routed into the main FK/IK paths. Optional PID motion is shown only as a secondary path for smooth target tracking.

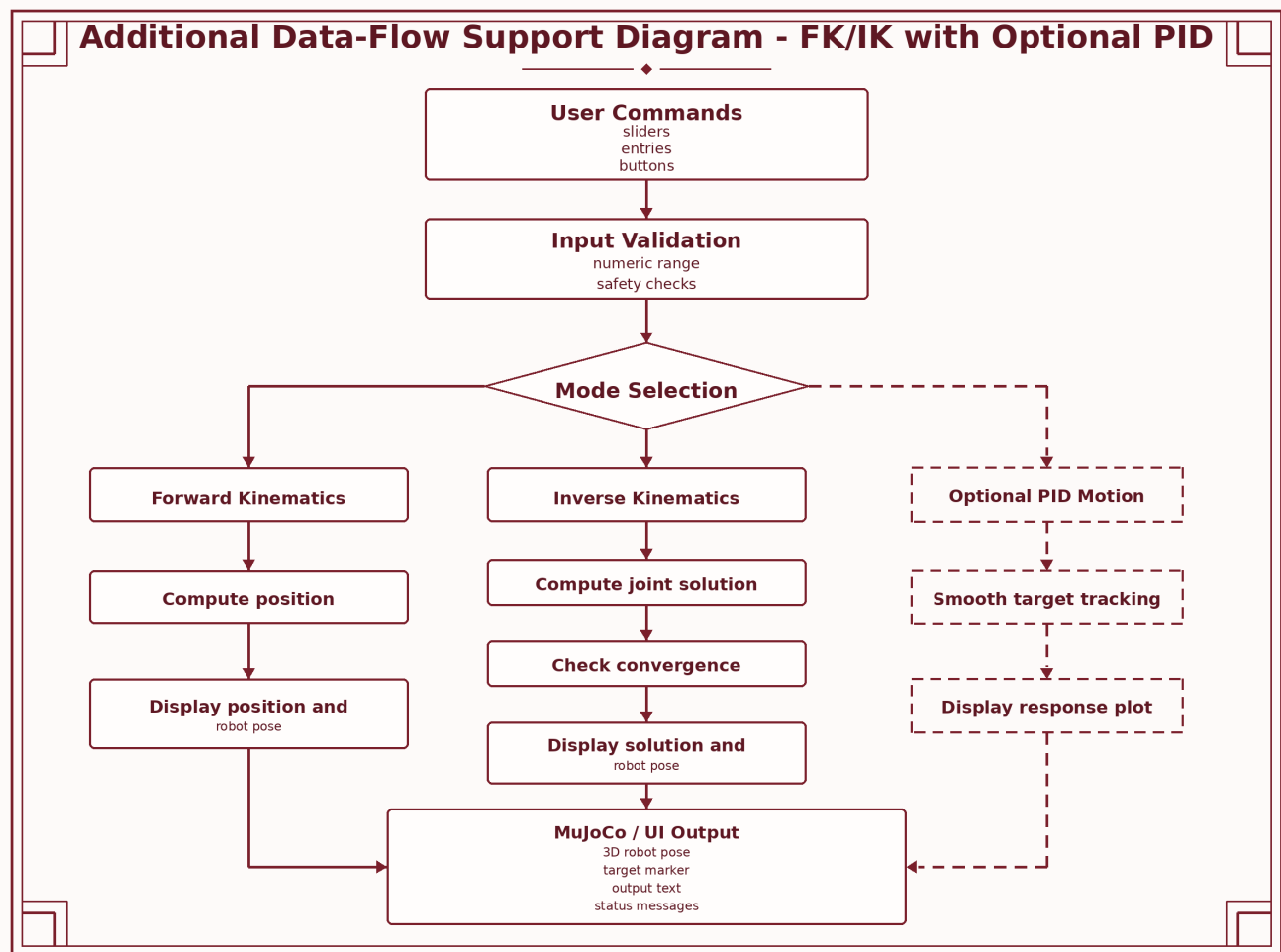


Figure 10: Additional data-flow support diagram for FK/IK with optional PID motion.

21. Bibliography / References

No.	Source	Use in Project
1	Main GitHub repository	Project source code, README, file structure, configuration, UI, FK/IK kinematics, optional PID, and MuJoCo implementation. https://github.com/joeljebitto-dev/School_cs_project_12th
2	Python documentation	Python language reference and standard library. https://docs.python.org/3/
3	Tkinter documentation	GUI framework reference. https://docs.python.org/3/library/tkinter.html
4	NumPy documentation	Arrays, trigonometry, norms, and linear algebra. https://numpy.org/doc/
5	Matplotlib documentation	Equation rendering and live PID plot. https://matplotlib.org/stable/
6	MuJoCo Python documentation	Simulation model, viewer, data, and stepping. https://mujoco.readthedocs.io/en/stable/python.html
7	MuJoCo PyPI	Installation reference for MuJoCo Python package. https://pypi.org/project/mujoco/
8	pytest documentation	Repository validation and checks. https://docs.pytest.org/

Scan for Main GitHub Repository



github.com/joeljebitto-dev/School_cs_project_12th

End of Project